Networks in R

EM1421 - Social media and web analytics

{Slides courtesy of Prof. Fabiana Zollo}

Prof. Pierluigi Zama Ramirez

Ca' Foscari University of Venice



igraph

- **igraph** is a library collection for creating, manipulating, and analyzing networks
- It is written in C and also exists as Python and R packages
- The software is widely used in academic research in network science and related fields

For more details on igraph visit https://igraph.org/redirect.html

A Basic Graph

- We want to represent parent-child relationships in a family tree (example: the Royal Family)
- We start by loading the `igraph` library and constructing an empty directed graph

```
1 library(igraph)
2 g <- graph.empty(directed=TRUE)
```

A Basic Graph

- Next, we need to manually add our data.
- First, each person in the family tree is added to the set of vertices
- Notice that we must reassign the new graph back into the variable g

```
g <- g + vertex("Elizabeth II")
g <- g + vertex("Philip")
g <- g + vertex("Charles")
g <- g + vertex("Diana")
g <- g + vertex("William")
g <- g + vertex("Harry")
g <- g + vertex("Catherine")
g <- g + vertex("George")
```

A Basic Graph

- We will also manually add edges to the graph for every parent-child relationship.
- As the graph is directed, it is important that each edge is created with the parent first and their child second.

```
g <- g + edges("Elizabeth II", "Charles")
g <- g + edges("Philip", "Charles")
g <- g + edges("Charles", "William")
g <- g + edges("Diana", "William")
g <- g + edges("Charles", "Harry")
g <- g + edges("Diana", "Harry")
g <- g + edges("William", "George")
g <- g + edges("Catherine", "George")
```

A Basic Graph

- We now have the complete structure of our family tree inside the R object g
- The process outlined here is fairly verbose
- For example, we can skip the step of manually adding each vertex
  - The R library would silently construct the nodes when adding edges

A Basic Graph

- Let's try to print the graph in the R console
- We can see that we have a directed (D), named (N), graph with 8 vertices and 8 edges

```
1 g
```

```
IGRAPH 5b3a165 DN-- 8 8 --
+ attr: name (v/c)
+ edges from 5b3a165 (vertex names):
[1] Elizabeth II->Charles Philip      ->Charles Charles
[4] Diana        ->William Charles    ->Harry   Diana
[7] William      ->George  Catherine  ->George
```

Edge, vertex, and network attributes

```
1 g
```

```
IGRAPH 5b3a165 DN-- 8 8 --
+ attr: name (v/c)
+ edges from 5b3a165 (vertex names):
[1] Elizabeth II->Charles Philip      ->Charles Charles
[4] Diana        ->William Charles    ->Harry   Diana
[7] William      ->George  Catherine  ->George
```

The description of an `igraph` object starts with up to 4 letters:

1. D or U, for a directed or undirected graph
2. N for a named graph (where nodes have a name attribute)
3. W for a weighted graph (where edges have a weight attribute)
4. B for a bipartite (two-mode) graph (where nodes have a type attribute)

Edge, vertex, and network attributes

```
1 g
```

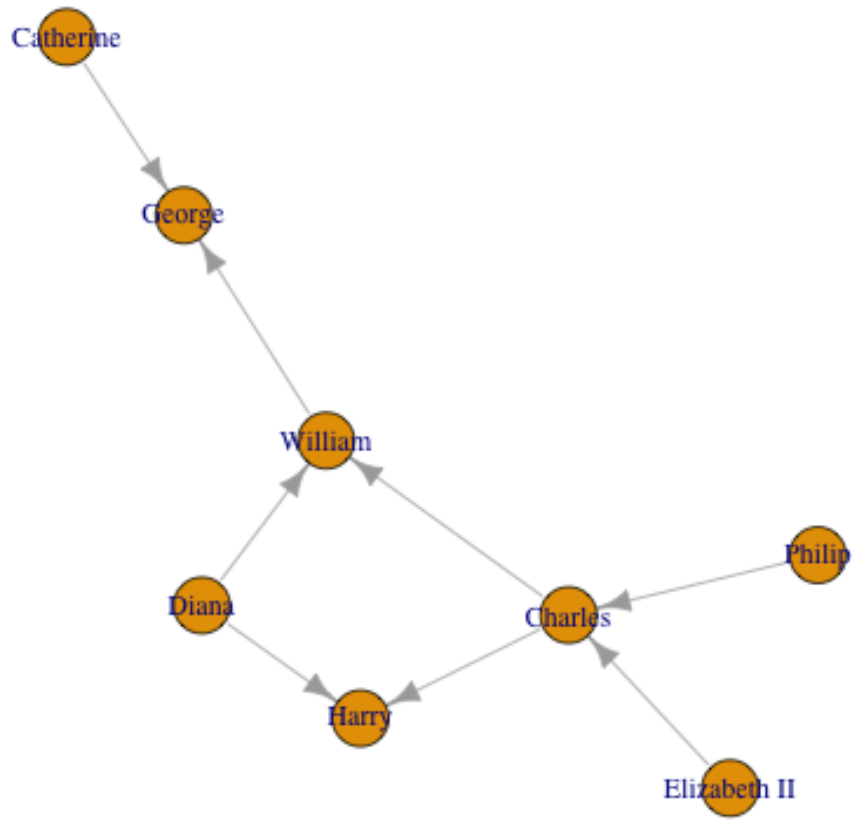
```
IGRAPH 5b3a165 DN-- 8 8 --
+ attr: name (v/c)
+ edges from 5b3a165 (vertex names):
[1] Elizabeth II->Charles Philip      ->Charles Charles
[4] Diana        ->William Charles    ->Harry   Diana
[7] William      ->George  Catherine  ->George
```

The description also lists node & edge attributes: * (g/c): graph-level character attribute * (v/c): vertex-level character attribute * (e/n): edge-level numeric attribute

A Basic Graph

If we plot the graph, we get something a bit more interesting

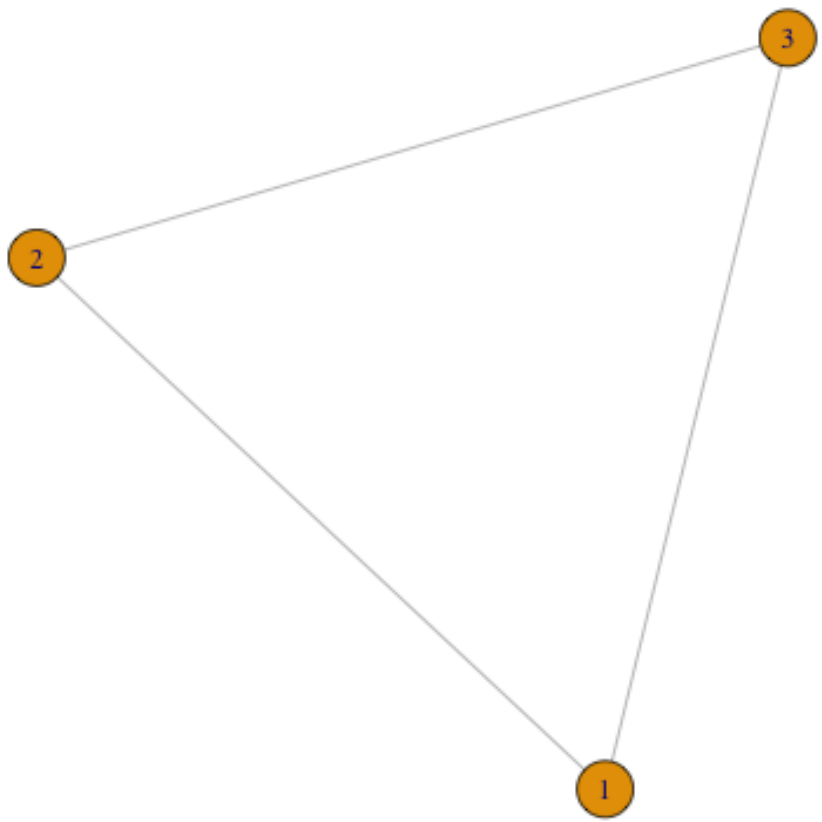
```
1 plot.igraph(g)
```



Create Networks

- The code below generates an undirected graph with three edges
- The numbers are interpreted as vertex IDs.

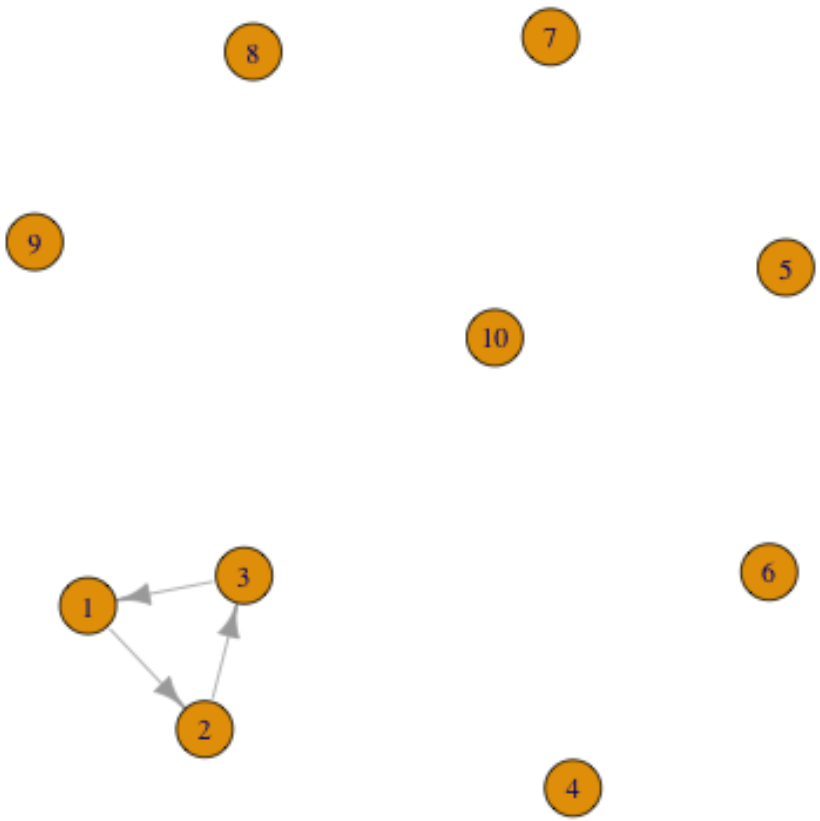
```
1  g1 <- graph( edges=c(1,2, 2,3, 3,1), n=3, directed=F); plot
```



Create Networks

Now with 10 vertices, and directed by default:

```
1 g2 <- graph( edges=c(1,2, 2,3, 3,1), n=10)
2 plot(g2)
```



Create Networks

```
1 g1
```

```
IGRAPH 6b57173 U--- 3 3 --
+ edges from 6b57173:
[1] 1--2 2--3 1--3
```

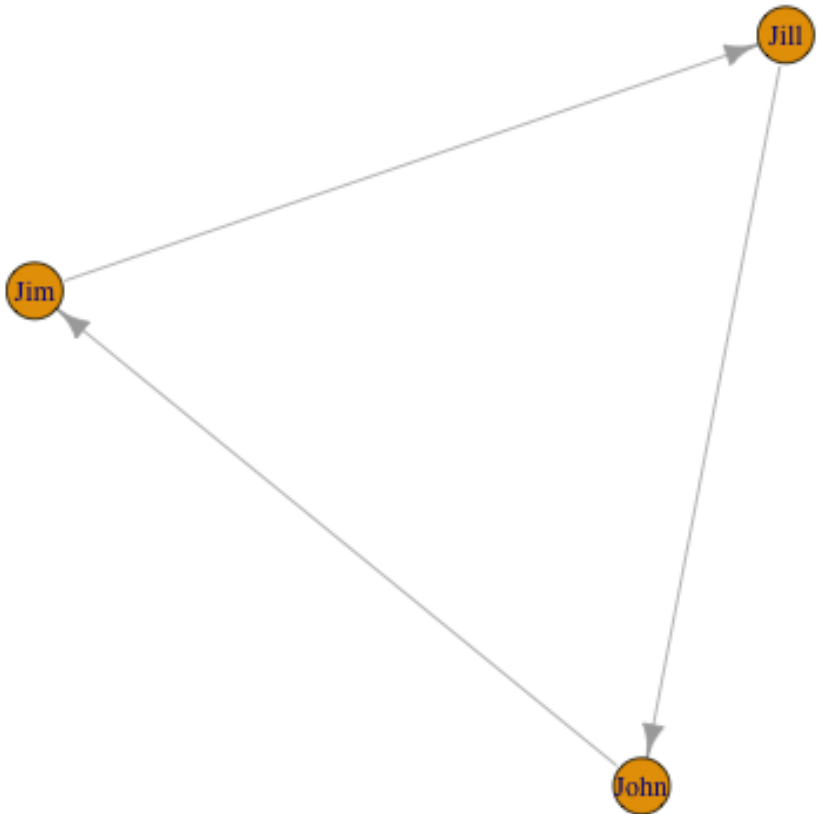
```
1 g2
```

```
IGRAPH aa0bce2 D--- 10 3 --
+ edges from aa0bce2:
[1] 1->2 2->3 3->1
```

Create Networks

When the edge list has vertex names, the number of nodes is not needed:

```
1 g3 <- graph( c("John", "Jim", "Jim", "Jill", "Jill", "John"
2 plot(g3)
```



Create Networks

```
g3
```

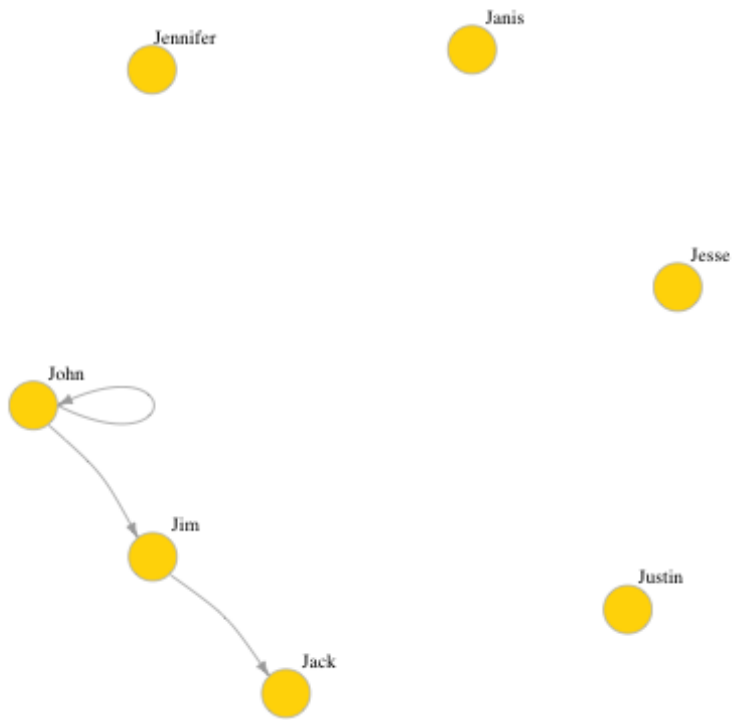
```
IGRAPH 2a3c2b1 DN-- 3 3 --
+ attr: name (v/c)
+ edges from 2a3c2b1 (vertex names):
[1] John->Jim  Jim ->Jill Jill->John
```

In named graphs we can specify isolates by providing a list of their names:

```
g4 <- graph( c("John", "Jim", "Jim", "Jack",
               "John", "John"),
             isolates=c("Jesse", "Janis",
                        "Jennifer", "Justin"))
```

Create Networks

```
plot(g4, edge.arrow.size=.5, vertex.color="gold",
     vertex.size=15, vertex.frame.color="gray",
     vertex.label.color="black", vertex.label.cex=0.8,
     vertex.label.dist=2, edge.curved=0.2)
```



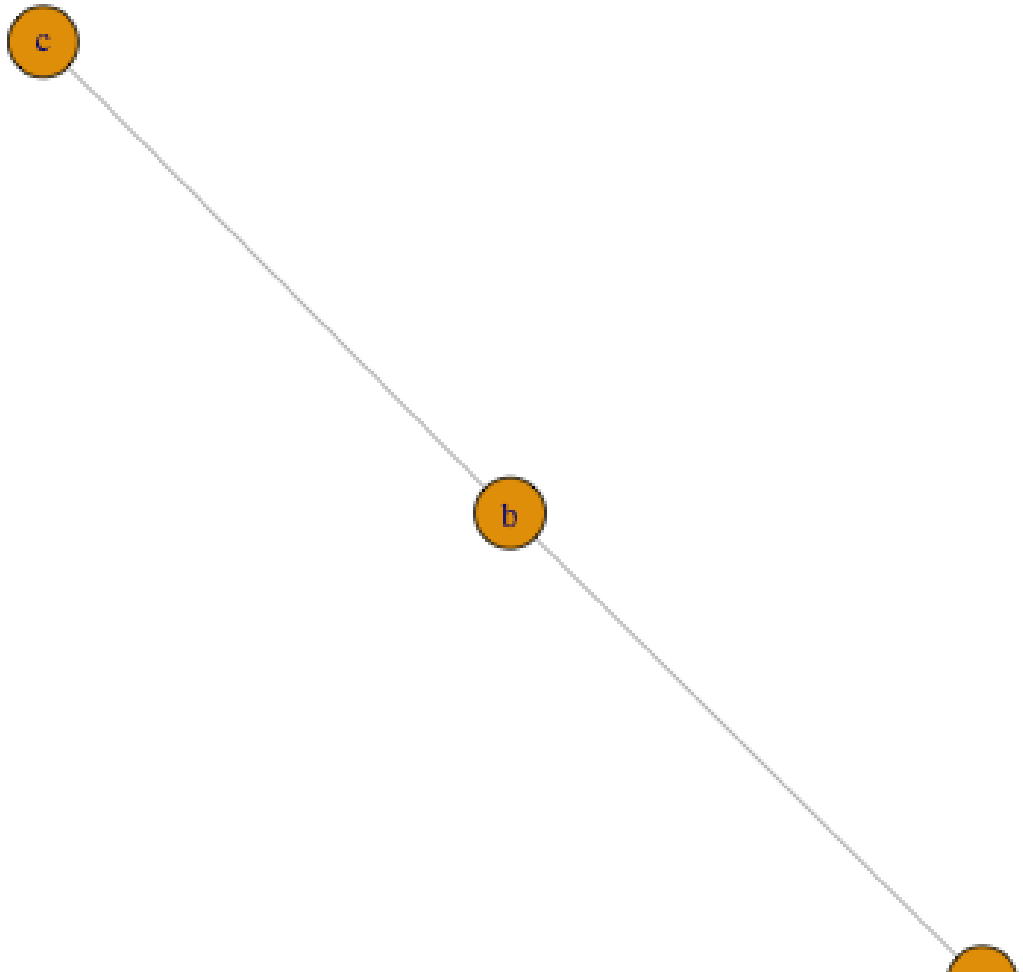
Create Networks

Small graphs can also be generated with a description of this kind:

- – for undirected tie
- +– or –+ for directed ties pointing left & right
- ++ for a symmetric tie
- : for sets of vertices

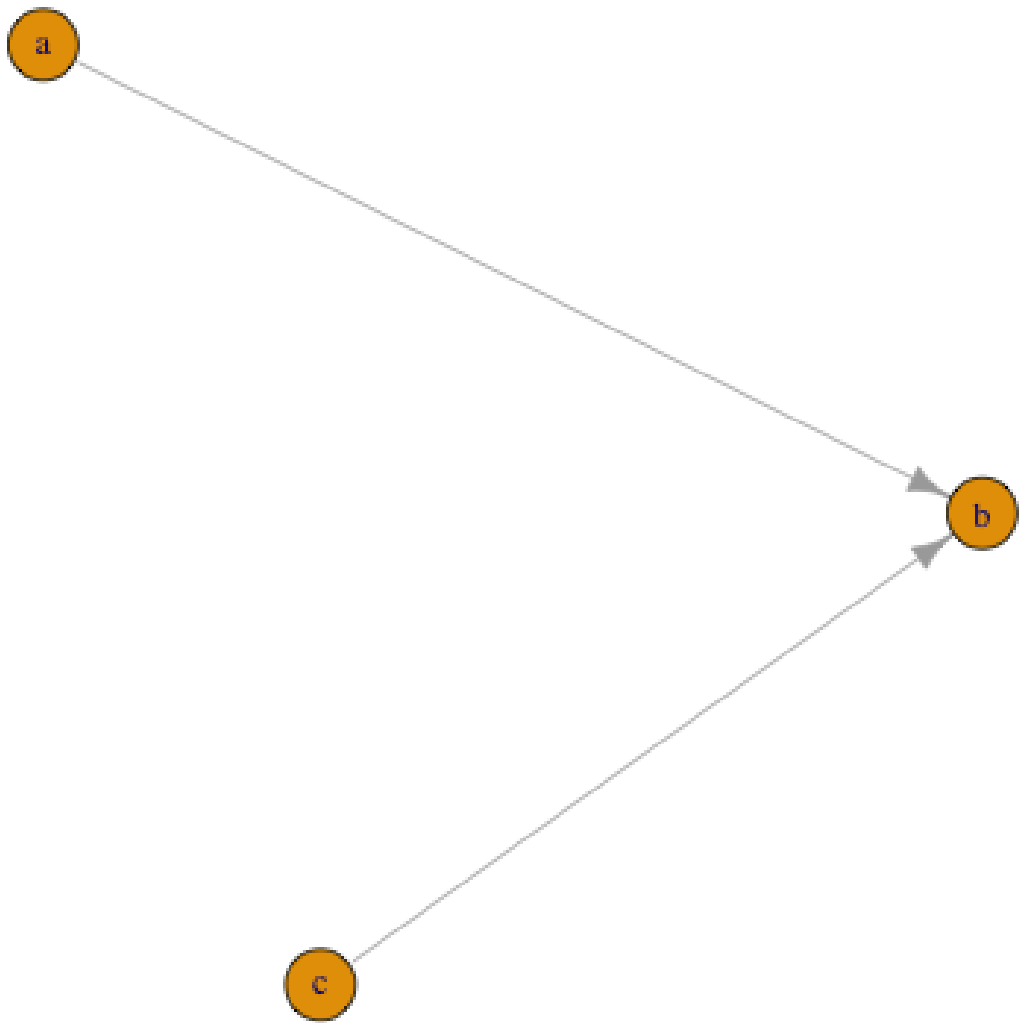
Create Networks

```
1 plot(graph_from_literal(a---b, b---c))
2 # the number of dashes doesn't matter
```



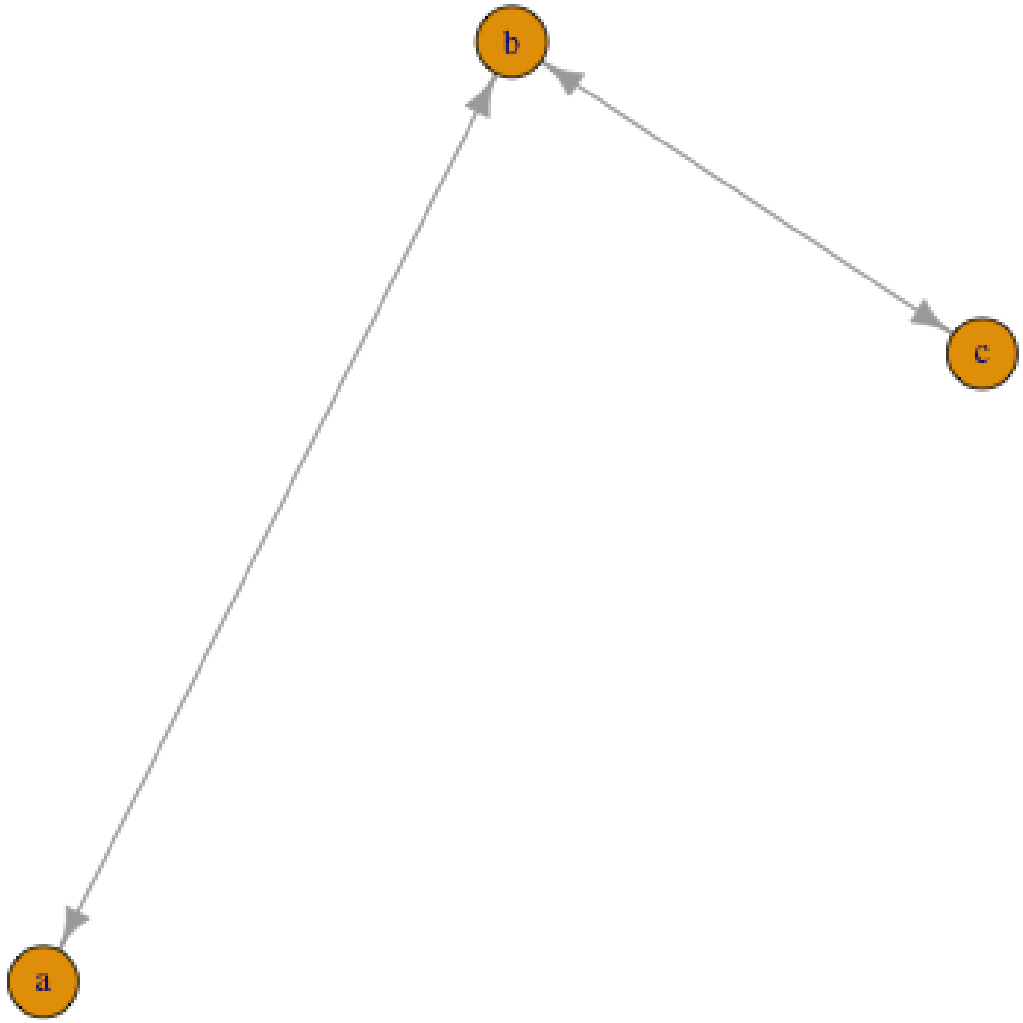
Create Networks

```
1 plot(graph_from_literal(a--+b, b+--c))
```



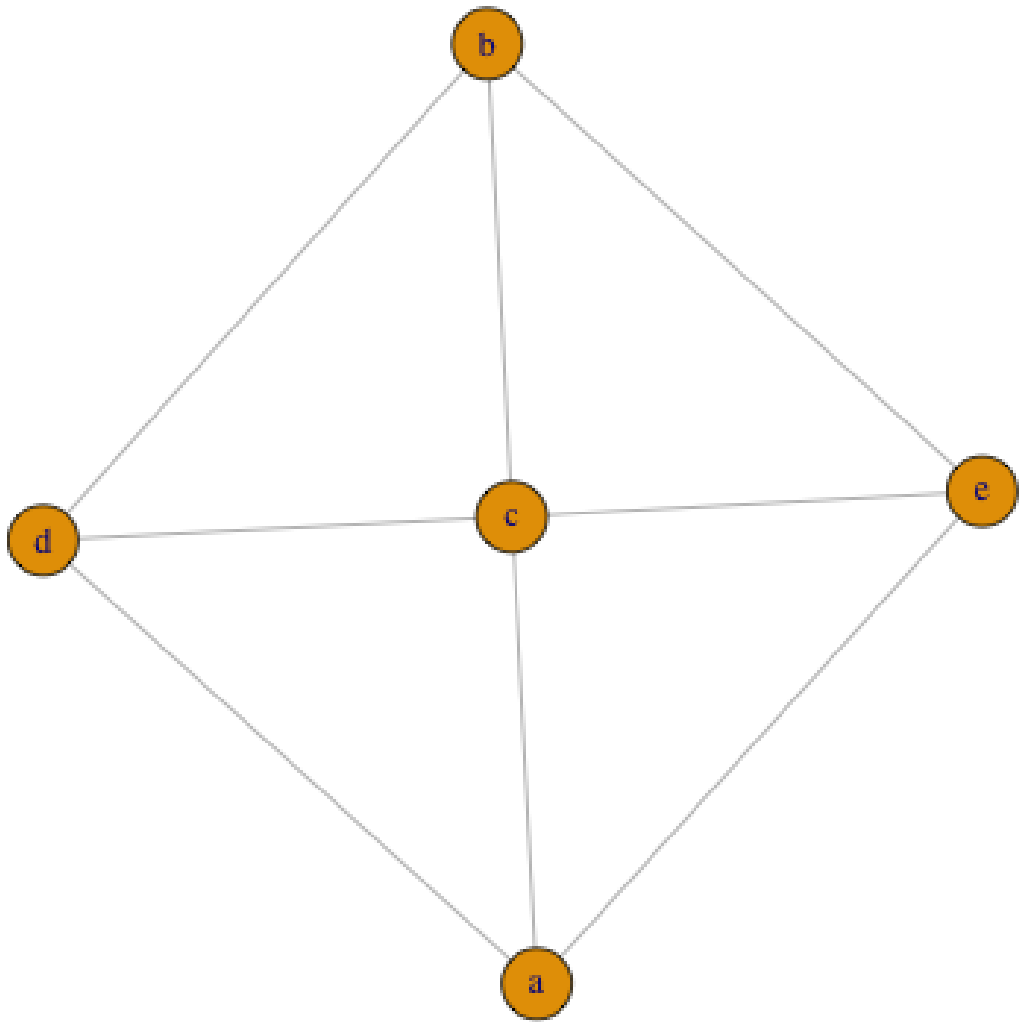
Create Networks

```
1 plot(graph_from_literal(a+-+b, b+-+c))
```



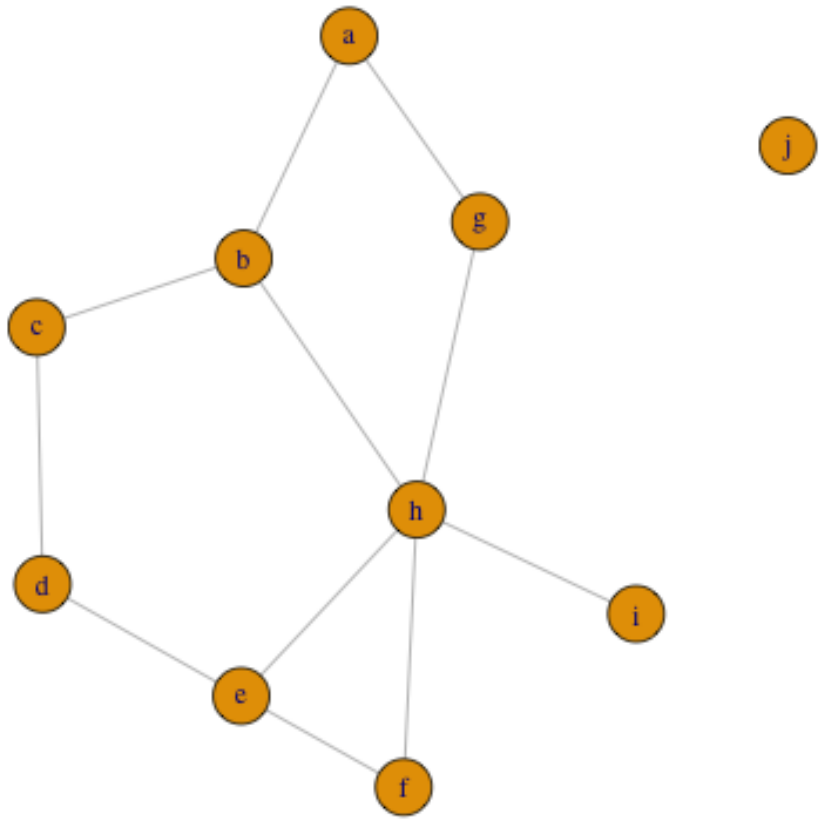
Create Networks

```
1 plot(graph_from_literal(a:b:c---c:d:e))
```

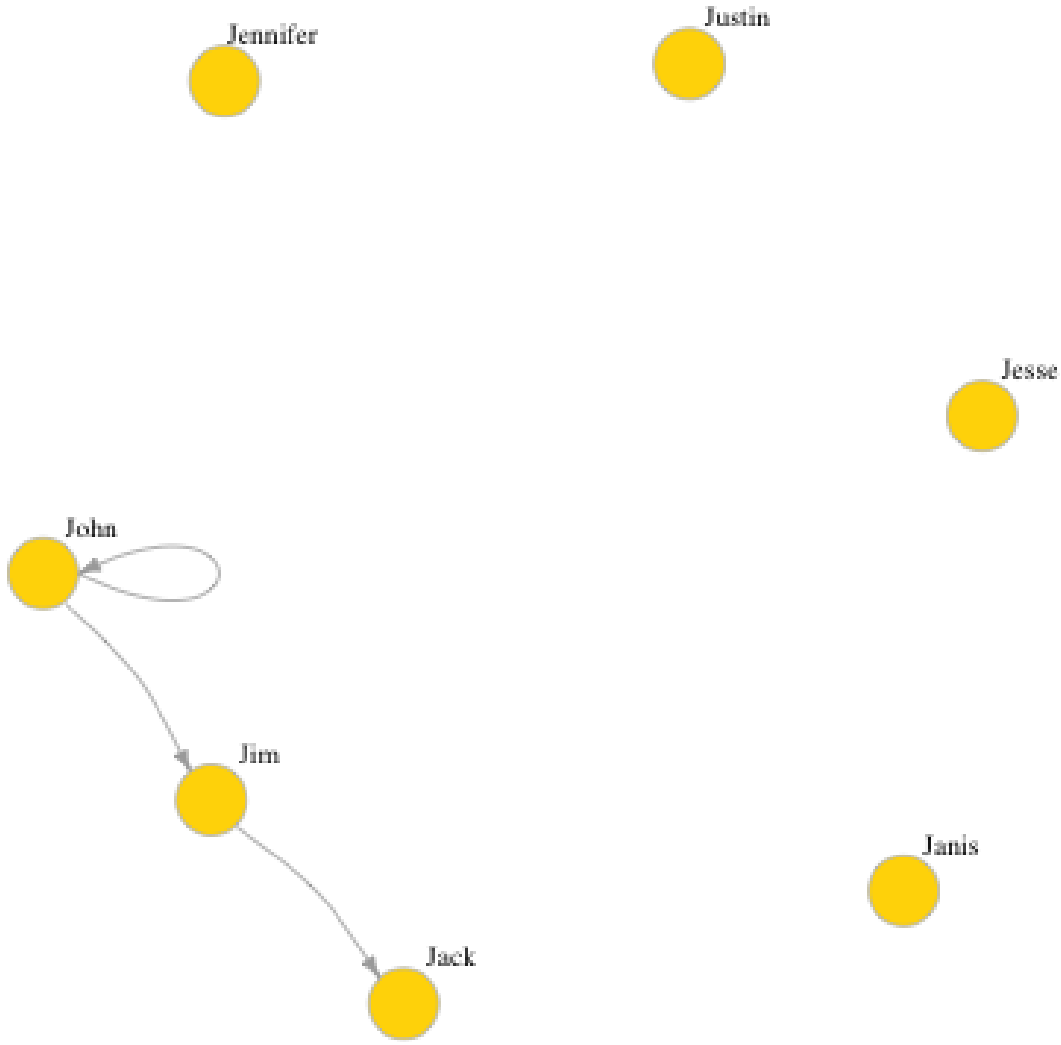


Create Networks

```
1 gl <- graph_from_literal(a-b-c-d-e-f, a-g-h-b, h-e:f:i, j)
2 plot(gl)
```



Edge, vertex, and network attributes



Edge, vertex, and network attributes

Access vertices and edges:

```
E(g4) # Edges of the object
```

```
+ 3/3 edges from c74b350 (vertex names):
[1] John->Jim  Jim ->Jack John->John
```

```
V(g4) # Vertices of the object
```

```
+ 7/7 vertices, named, from c74b350:
[1] John  Jim  Jack  Jesse Janis Jennifer Justin
```

Edge, vertex, and network attributes

You can also examine the network matrix directly:

```
1 g4[]
```

```
7 x 7 sparse Matrix of class "dgCMatrix"
         John Jim Jack Jesse Janis Jennifer Justin
John        1   1    .     .     .        .      .
Jim         .   .    1     .     .        .      .
Jack        .   .    .     .     .        .      .
Jesse       .   .    .     .     .        .      .
Janis       .   .    .     .     .        .      .
Jennifer    .   .    .     .     .        .      .
Justin      .   .    .     .     .        .      .
```

Edge, vertex, and network attributes

You can access attributes of the network, vertices, or edges:

```
V(g4)$name
#automatically generated when we created the network
```

```
[1] "John" "Jim" "Jack" "Jesse" "Janis" "Jennifer" "Justin"
```

Edge, vertex, and network attributes

And add new ones:

```
V(g4)$status <- c("intern", "intern", "intern",
                  "staff", "staff", "staff",
                  "staff")

E(g4)$type <- "email"
#Edge attribute, assign "email" to all edges

E(g4)$weight <- 10
# Edge weight, setting all existing edges to 10
```

Edge, vertex, and network attributes

Examine edge attributes:

```
1 edge_attr(g4)
```

```
$type
[1] "email" "email" "email"

$weight
[1] 10 10 10
```

Edge, vertex, and network attributes

Examine vertex attributes:

```
1 vertex_attr(g4)
```

```
$name
[1] "John" "Jim" "Jack" "Jesse" "Janis"
[2] "Jennifer" "Justin"

$status
[1] "intern" "intern" "intern" "staff" "staff"
[2] "staff" "staff"
```

Edge, vertex, and network attributes

Another way to set attributes:

```
g4 <- set_graph_attr(g4, "name", "Email Network")
graph_attr_names(g4)
```

```
[1] "name"
```

```
g4 <- set_graph_attr(g4, "company", "Microsoft")
graph_attr_names(g4)
```

```
[1] "name"    "company"
```

Similarly, you can use `set_edge_attr()`, `set_vertex_attr()`, etc.

Edge, vertex, and network attributes

```
graph_attr(g4, "name")
```

```
[1] "Email Network"
```

```
graph_attr(g4)
```

```
$name
[1] "Email Network"

$company
[1] "Microsoft"
```

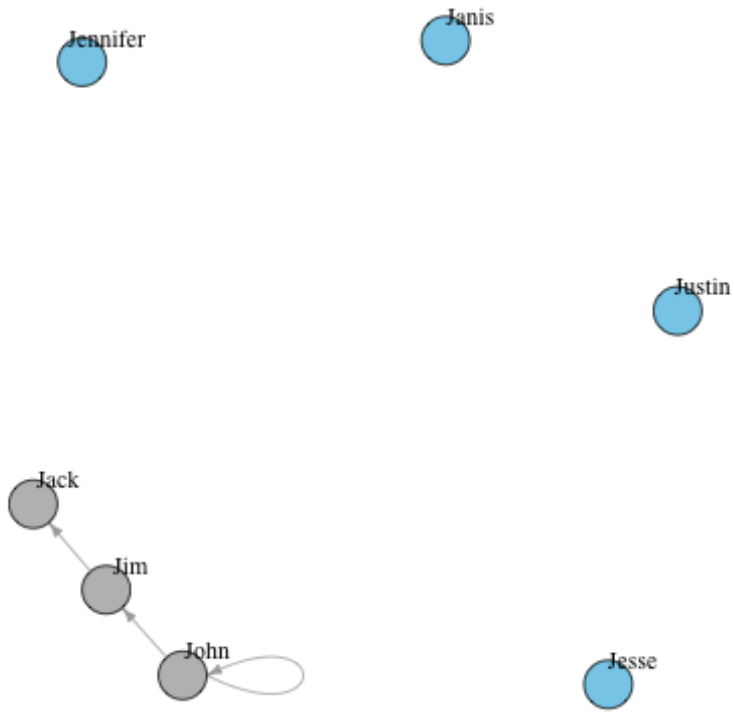
Edge, vertex, and network attributes

```
g4 <- delete_graph_attr(g4, "company")
graph_attr(g4)
```

```
$name
[1] "Email Network"
```

Edge, vertex, and network attributes

```
plot(g4, edge.arrow.size=.5, vertex.label.color="black",
      vertex.label.dist=1.5,
      vertex.color =
        c( "skyblue", "grey")[1+(V(g4)$status=="intern")])
```

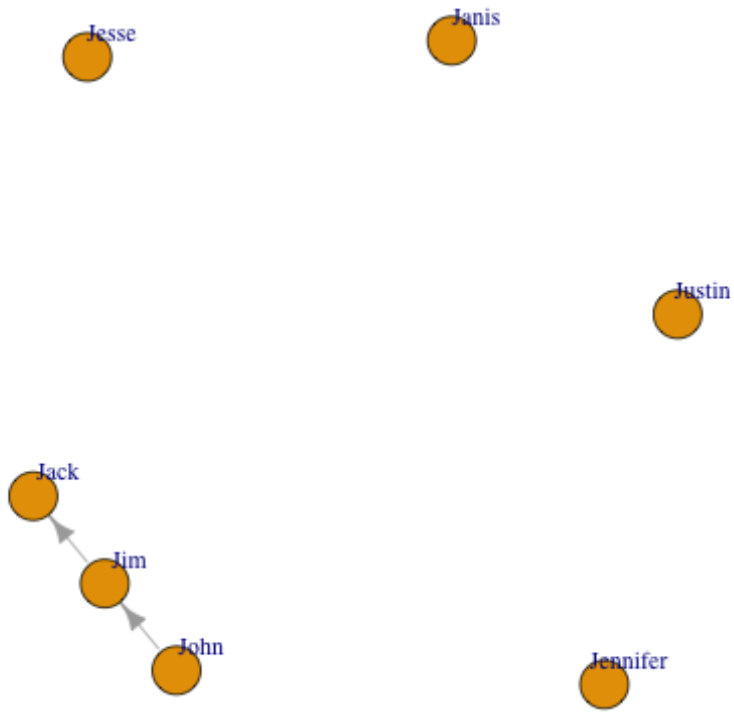


Edge, vertex, and network attributes

- The graph `g4` has a loop from John to himself
- We can simplify our graph to remove loops & multiple edges between the same nodes
- Use `edge.attr.comb` to indicate how edge attributes are to be combined (possible options include `sum`, `mean`, `min`, `max`)
- Option `ignore` says the attribute should be disregarded and dropped

Edge, vertex, and network attributes

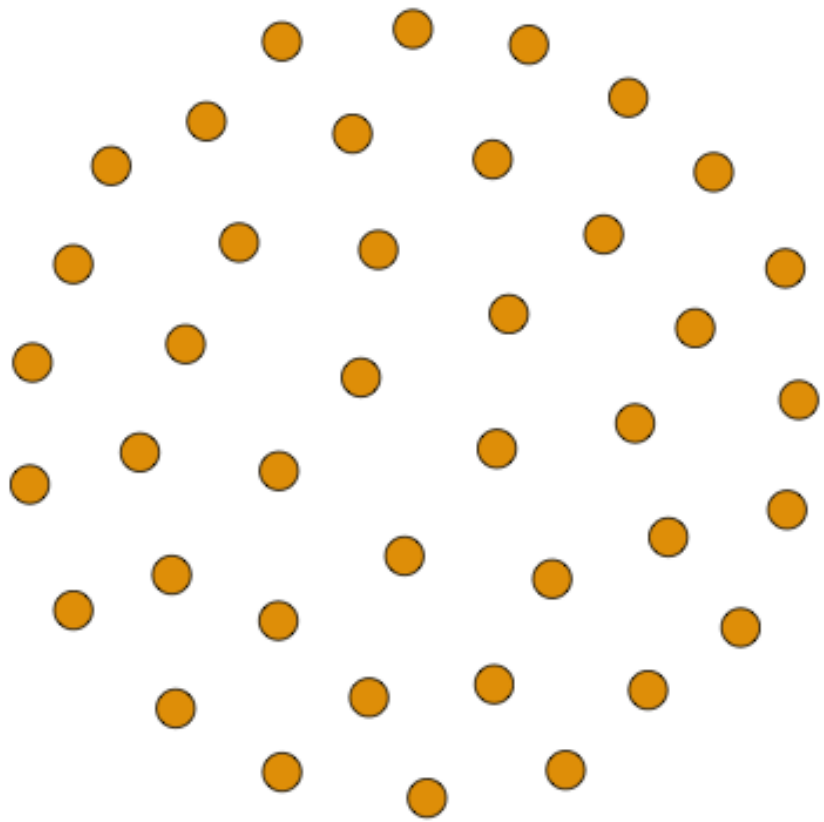
```
g4s <- simplify( g4, remove.loops = T,
                 edge.attr.comb=c(weight="sum",
                                  type="ignore") )
plot(g4s, vertex.label.dist=1.5)
```



Specific graphs

Empty graph

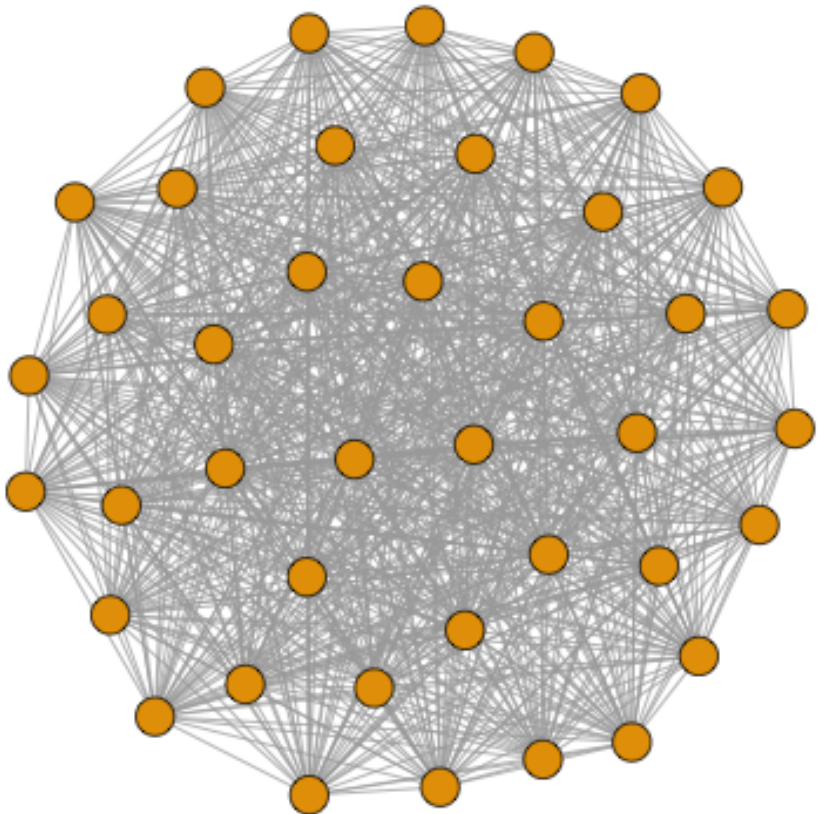
```
1 eg <- make_empty_graph(40)
2 plot(eg, vertex.size=10, vertex.label=NA)
```



Specific graphs

A **complete graph** is a simple undirected graph in which every pair of distinct vertices is connected by a unique edge:

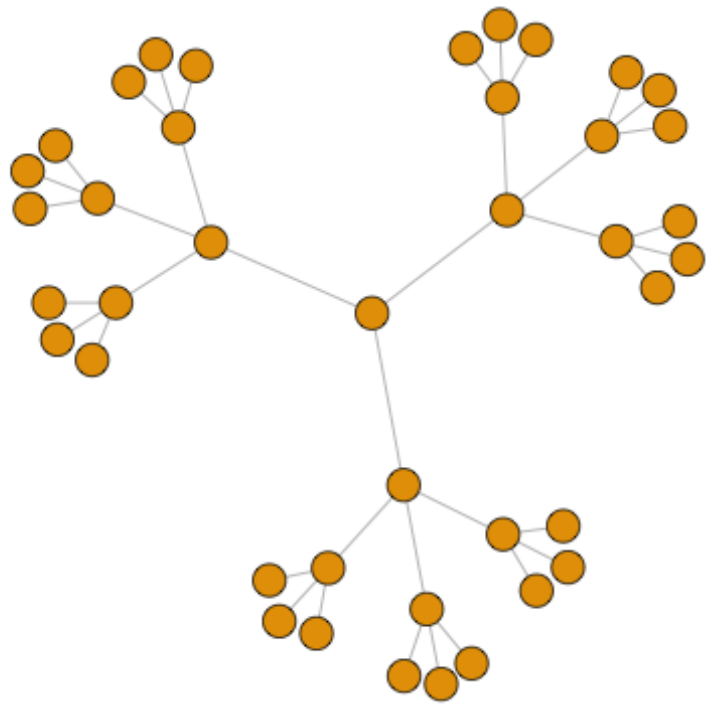
```
1 fg <- make_full_graph(40)
2 plot(fg, vertex.size=10, vertex.label=NA)
```



Specific graphs and graph models

A **tree** is an undirected graph in which any two vertices are connected by exactly one path, or equivalently a connected acyclic undirected graph

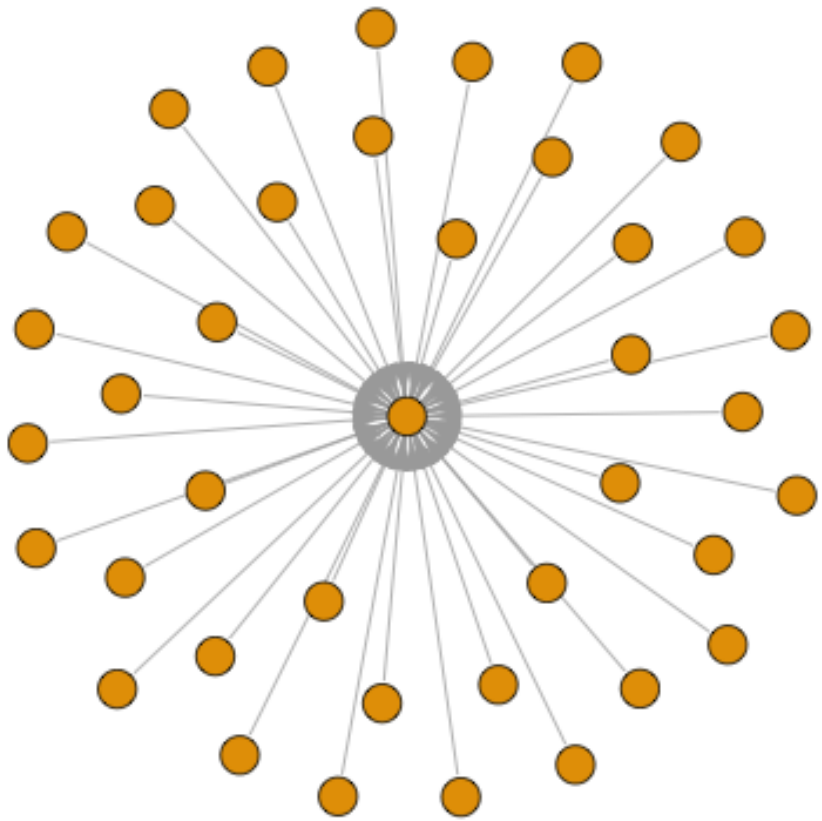
```
1 tr <- make_tree(40, children = 3, mode = "undirected")
2 plot(tr, vertex.size=10, vertex.label=NA)
```



Specific graphs and graph models

A **star graph** is a tree with one internal node and k leaves:

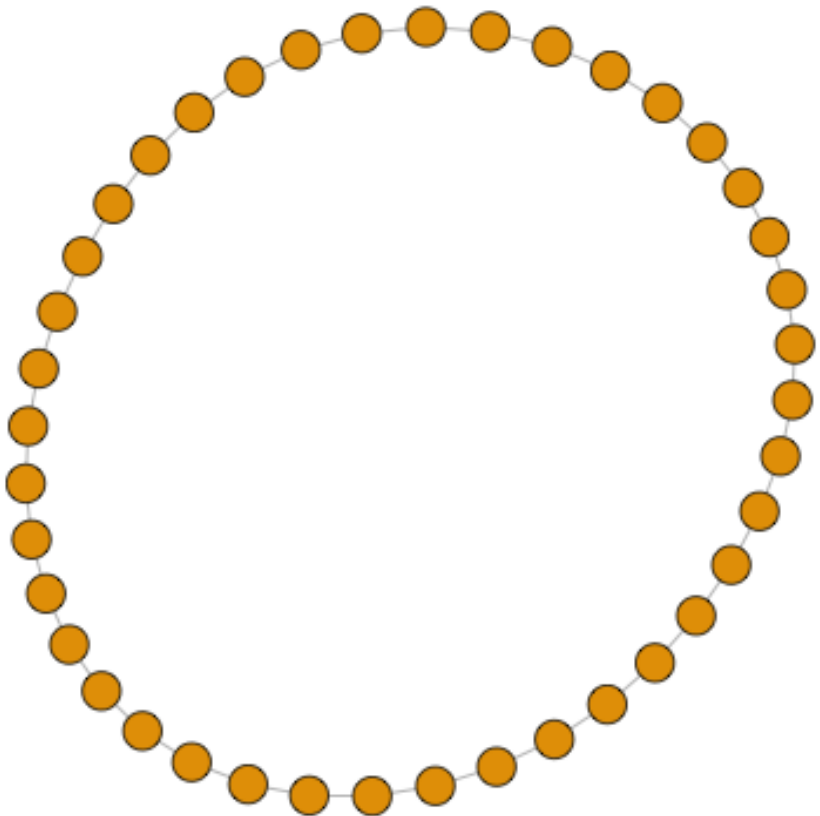
```
st <- make_star(40)
plot(st, vertex.size=10, vertex.label=NA)
```



Specific graphs

A **ring graph** is an undirected graph whose vertices are arranged in a ring so that each vertex has exactly two neighbors

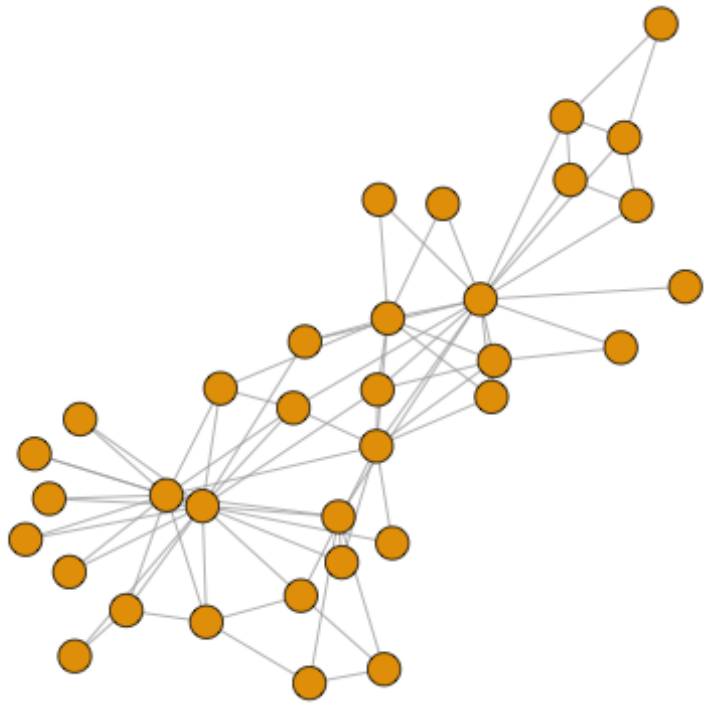
```
1 rn <- make_ring(40)
2 plot(rn, vertex.size=10, vertex.label=NA)
```



Specific graphs

`igraph` can also give you some notable historical graphs. For instance:

```
1 zach <- graph("Zachary") # the Zachary Karate Club
2 plot(zach, vertex.size=10, vertex.label=NA)
```



Diameter

A network diameter is the length of the shortest path between two nodes in the network:

```
diameter(zach, directed=F)
```

```
[1] 5
```

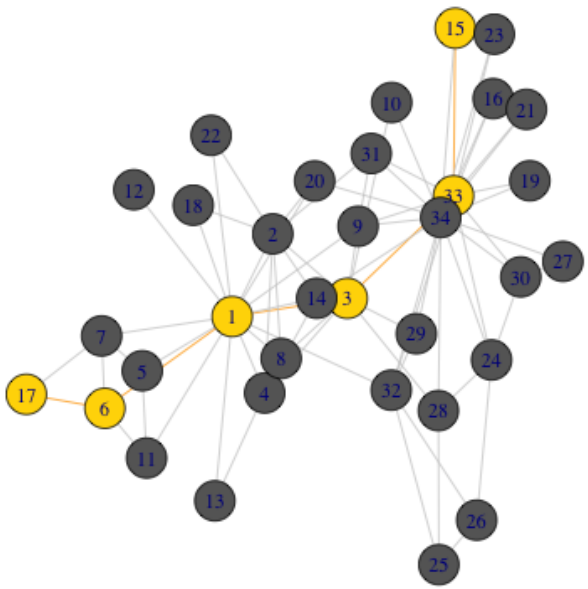
```
diam <- get_diameter(zach, directed=F)
diam
```

```
+ 6/34 vertices, from b50c1ee:
[1] 15 33  3  1  6 17
```

Diameter

Color nodes along the diameter:

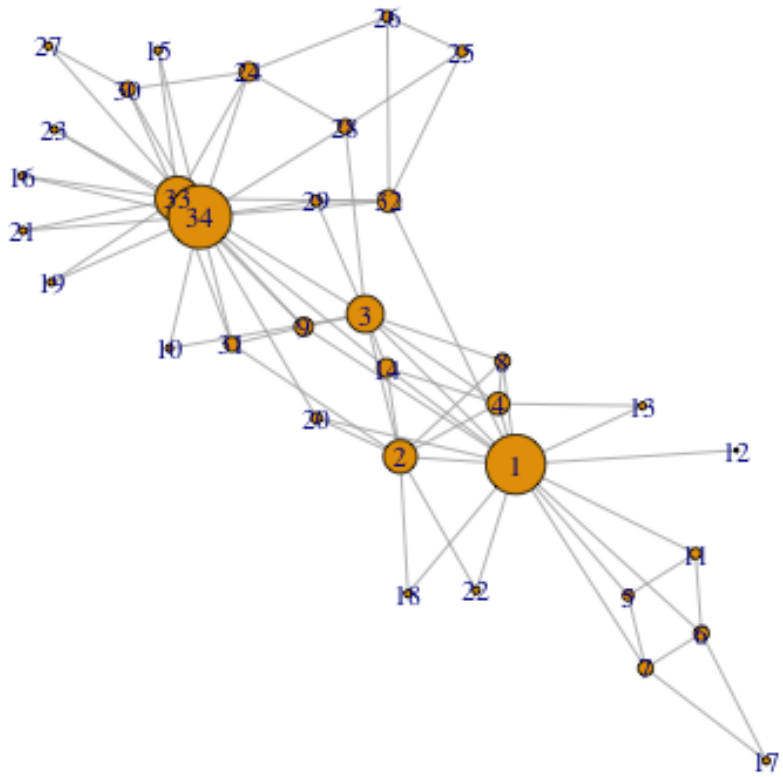
```
vcol <- rep("gray40", vcount(zach))
vcol[diam] <- "gold"
ecol <- rep("gray80", ecount(zach))
ecol[E(zach, path=diam)] <- "orange"
plot(zach, vertex.color=vcol, edge.color=ecol,
     edge.arrow.mode=0)
```



Node degree

The function `degree()` has a mode of `in` for in-degree, `out` for out-degree, and `all` or `total` for total degree;

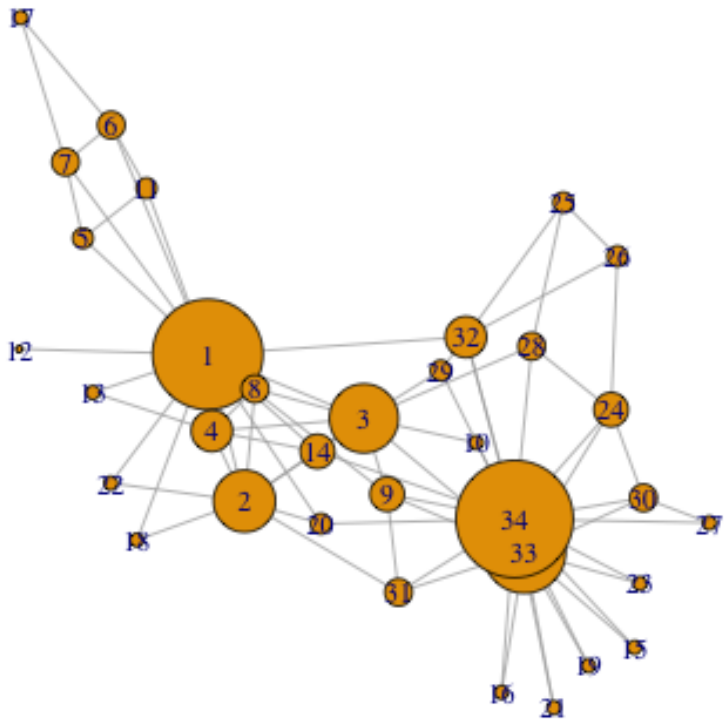
```
1 deg <- degree(zach, mode="all")
2 plot(zach, vertex.size=deg)
```



Node degree

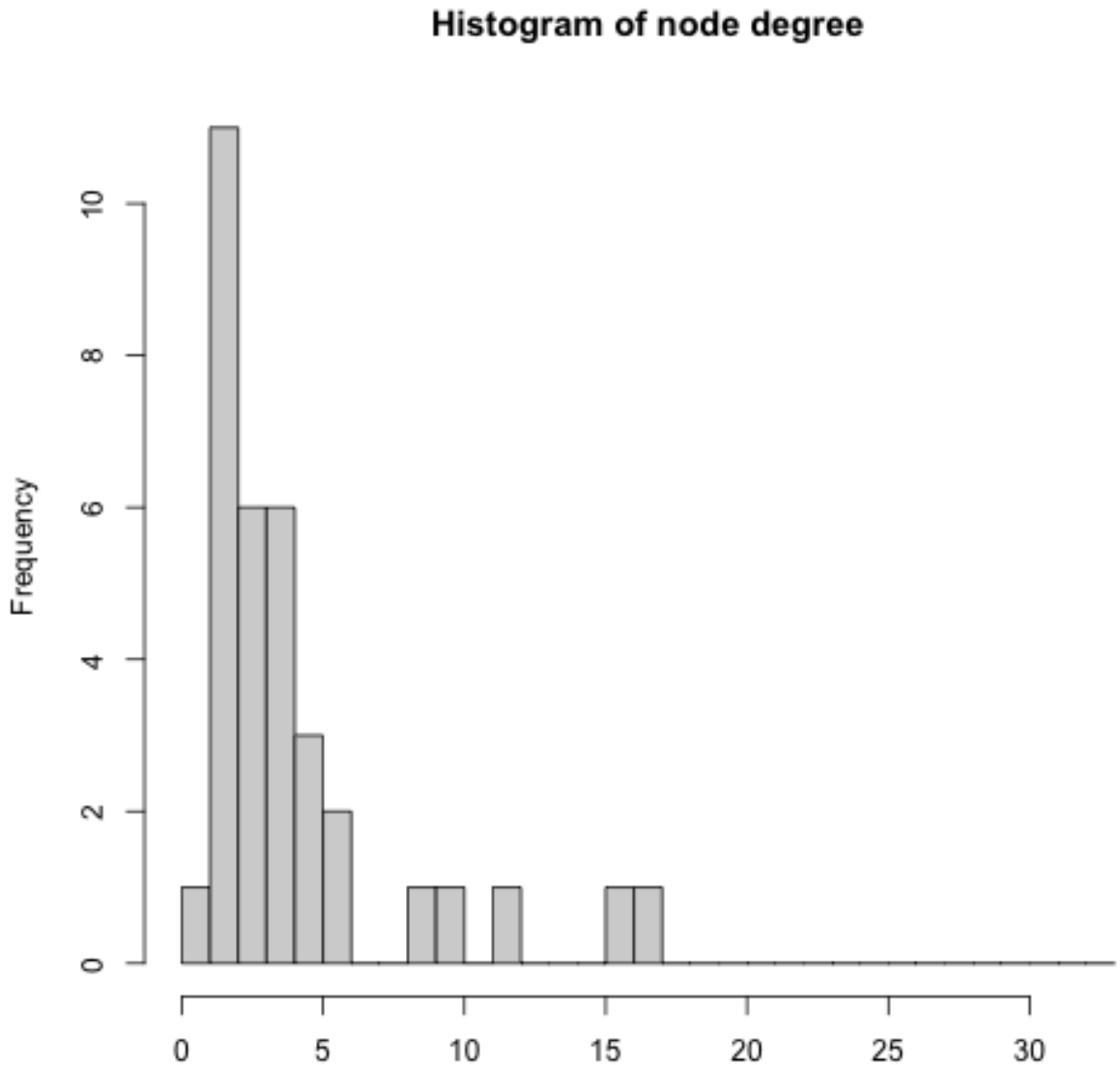
The function `degree()` has a mode of `in` for in-degree, `out` for out-degree, and `all` or `total` for total degree;

```
1 deg <- degree(zach, mode="all")
2 plot(zach, vertex.size=deg*2)
```



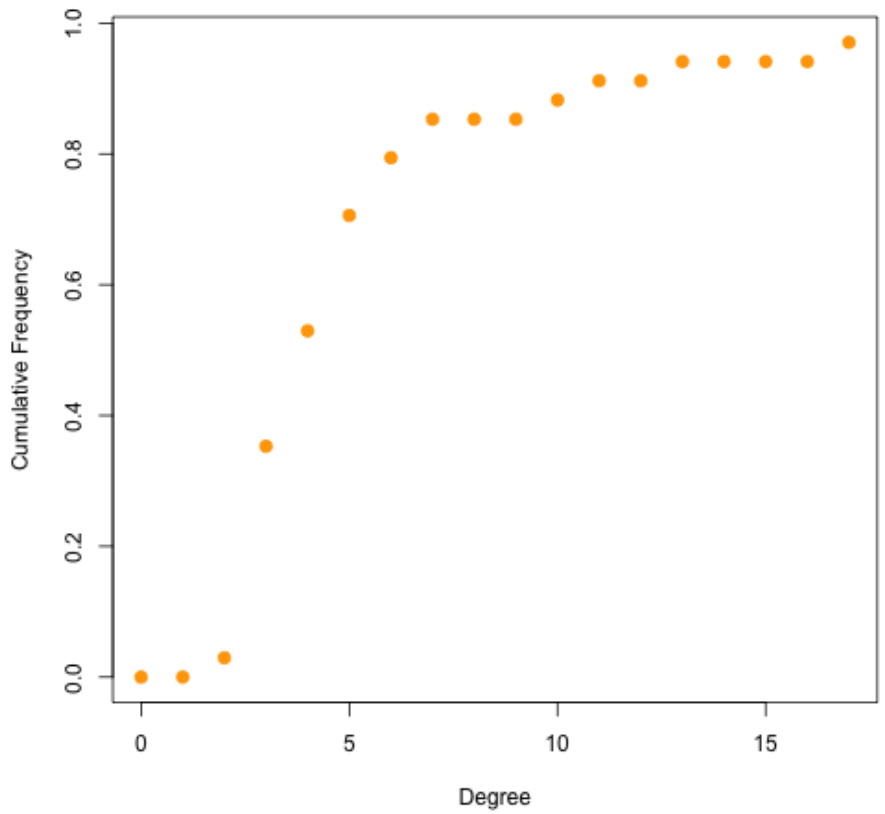
Node degree

```
1 hist(deg, breaks=1:vcount(zach)-1,
2       main="Histogram of node degree")
```



Degree distribution

```
1 deg.dist <- degree_distribution(zach, cumulative=T,
2                                 mode="all")
3 plot( x=0:max(deg), y=1-deg.dist, pch=19, cex=1.2,
4       col="orange", xlab="Degree",
5       ylab="Cumulative Frequency")
```



Distances and paths

Average path length, the mean of the shortest distance between each pair of nodes in the network

```
1 mean_distance(zach, directed=F)
```

```
[1] 2.4082
```

Distances and paths

We can also find the length of all shortest paths in the graph:

```
1 distances(zach)
```

```
      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10] [
 [1,]    0    1    1    1    1    1    1    1    1     2
 [2,]    1    0    1    1    2    2    2    1    2     2
 [3,]    1    1    0    1    2    2    2    1    1     1
 [4,]    1    1    1    0    2    2    2    1    2     2
 [5,]    1    2    2    2    0    2    1    2    2     3
 [6,]    1    2    2    2    2    0    1    2    2     3
 [7,]    1    2    2    2    1    1    0    2    2     3
 [8,]    1    1    1    1    2    2    2    0    2     2
 [9,]    1    2    1    2    2    2    2    2    0     2
[10,]    2    2    1    2    3    3    3    2    2     0
[11,]    1    2    2    2    1    1    2    2    2     3
[12,]    1    2    2    2    2    2    2    2    2     3
[13,]    1    2    2    1    2    2    2    2    2     3
[14,]    1    1    1    1    2    2    2    2    2     2
```

Reading network data from files: edge list

- Reading edge list (i -> j)

```
nodes <- read.csv("Dataset1-Media-Example-NODES.csv",
                  header=T, as.is=T)
links <- read.csv("Dataset1-Media-Example-EDGES.csv",
                  header=T, as.is=T)
```

Examine the data

```
1 head(nodes)
```

```
   id                media media.type type.label audience.si
1 s01             NY Times          1  Newspaper
2 s02      Washington Post          1  Newspaper
3 s03 Wall Street Journal           1  Newspaper
4 s04            USA Today          1  Newspaper
5 s05             LA Times          1  Newspaper
6 s06        New York Post          1  Newspaper
```

```
1 head(links)
```

```
  from  to weight      type
1  s01 s02     10 hyperlink
2  s01 s02     12 hyperlink
3  s01 s03     22 hyperlink
4  s01 s04     21 hyperlink
5  s04 s11     22   mention
6  s05 s15     21   mention
```

Examine the data

```
nrow(nodes);# OR length(unique(nodes$id))
```

```
[1] 17
```

```
nrow(links);# OR nrow(unique(links[,c("from", "to")]))
```

```
[1] 52
```

Weighted Networks

- Collapse multiple links of the same type between the same two nodes

```
links <- aggregate(links[,3], by=links[,-3], sum)
#3rd col = weight
links <- links[order(links$from, links$to),]
colnames(links)[4] <- "weight"
rownames(links) <- NULL
```

Turning networks into igraph objects

- Converting the data to an igraph object:
- The graph.data.frame function takes two data frames: 'd' and 'vertices'
  - 'd' describes the edges of the network - it should start with two columns containing the source and target node IDs for each network tie
  - 'vertices' should start with a column of node IDs
- Any additional columns in either data frame are interpreted as attributes

Turning networks into igraph objects

```
net <- graph_from_data_frame(d=links,
                             vertices=nodes, directed=T)
class(net)
```

```
[1] "igraph"
```

```
net
```

```
IGRAPH d1f460f DNW- 17 49 --
+ attr: name (v/c), media (v/c), media.type (v/n), type.lab
| audience.size (v/n), type (e/c), weight (e/n)
+ edges from d1f460f (vertex names):
 [1] s01->s02 s01->s03 s01->s04 s01->s15 s02->s01 s02->s03
 [9] s03->s01 s03->s04 s03->s05 s03->s08 s03->s10 s03->s11
[17] s04->s06 s04->s11 s04->s12 s04->s17 s05->s01 s05->s02
[25] s06->s06 s06->s16 s06->s17 s07->s03 s07->s08 s07->s10
[33] s08->s07 s08->s09 s09->s10 s10->s03 s12->s06 s12->s13
[41] s13->s17 s14->s11 s14->s13 s15->s01 s15->s04 s15->s06
[49] s17->s04
```

Turning networks into igraph objects

- Edges and Vertices

```
1 E(net)
```

```
+ 49/49 edges from d1f460f (vertex names):
 [1] s01->s02 s01->s03 s01->s04 s01->s15 s02->s01 s02->s03
 [9] s03->s01 s03->s04 s03->s05 s03->s08 s03->s10 s03->s11
[17] s04->s06 s04->s11 s04->s12 s04->s17 s05->s01 s05->s02
[25] s06->s06 s06->s16 s06->s17 s07->s03 s07->s08 s07->s10
[33] s08->s07 s08->s09 s09->s10 s10->s03 s12->s06 s12->s13
[41] s13->s17 s14->s11 s14->s13 s15->s01 s15->s04 s15->s06
[49] s17->s04
```

```
1 V(net)
```

```
+ 17/17 vertices, named, from d1f460f:
 [1] s01 s02 s03 s04 s05 s06 s07 s08 s09 s10 s11 s12 s13 s1
```

Turning networks into igraph objects

- Edges and Vertices Attributes

```
E(net)$type
```

```
 [1] "hyperlink" "hyperlink" "hyperlink" "mention"   "hyper
 [7] "hyperlink" "hyperlink" "hyperlink" "hyperlink" "hyper
[13] "mention"   "hyperlink" "hyperlink" "hyperlink" "menti
[19] "hyperlink" "mention"   "mention"   "hyperlink" "hyper
[25] "hyperlink" "hyperlink" "mention"   "mention"   "menti
[31] "mention"   "hyperlink" "mention"   "mention"   "menti
[37] "mention"   "hyperlink" "mention"   "hyperlink" "menti
[43] "mention"   "hyperlink" "hyperlink" "hyperlink" "hyper
[49] "hyperlink"
```

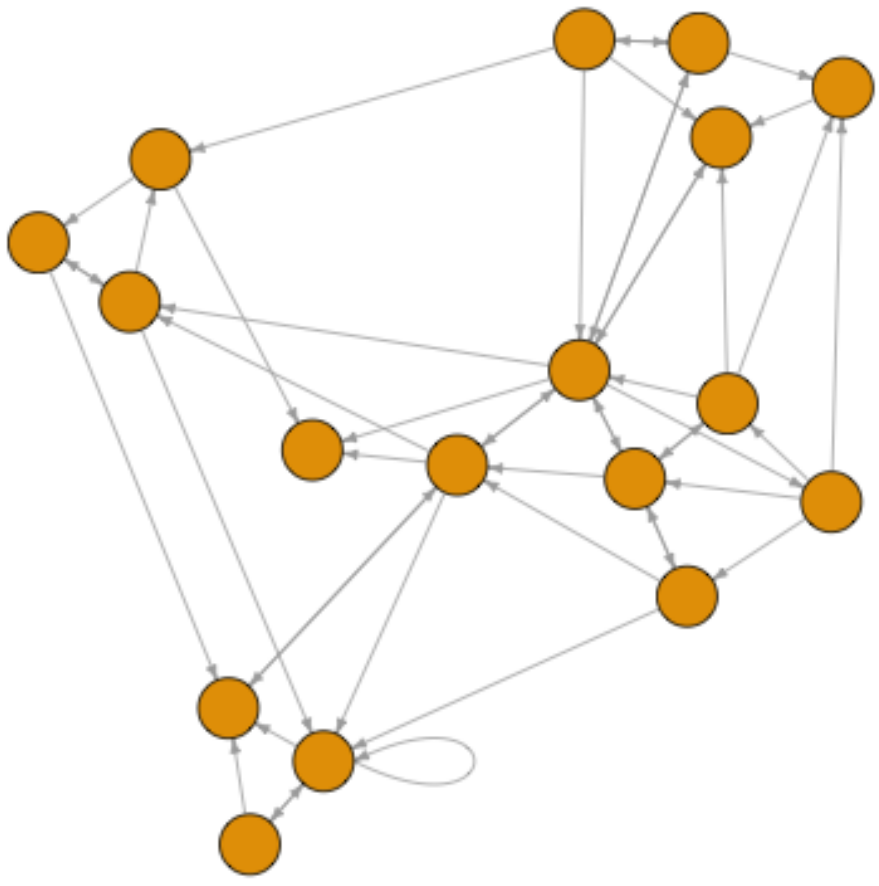
```
V(net)$media
```

```
 [1] "NY Times"          "Washington Post"     "Wall Stre
 [4] "USA Today"         "LA Times"            "New York
 [7] "CNN"               "MSNBC"               "FOX News"
[10] "ABC"               "BBC"                 "Yahoo New
```

Turning networks into igraph objects

- Plotting the network

```
1  plot(net, edge.arrow.size=.4,vertex.label=NA)
```



Turning networks into igraph objects

```
# Removing loops from the graph:
net <- simplify(net, remove.multiple = F, remove.loops = T)

# If you need them, you can extract an edge list
as_edgelist(net, names=T)
```

```
      [,1]  [,2]
 [1,] "s01" "s02"
 [2,] "s01" "s03"
 [3,] "s01" "s04"
 [4,] "s01" "s15"
 [5,] "s02" "s01"
 [6,] "s02" "s03"
 [7,] "s02" "s09"
 [8,] "s02" "s10"
 [9,] "s03" "s01"
[10,] "s03" "s04"
[11,] "s03" "s05"
[12,] "s03" "s08"
```

Turning networks into igraph objects

```
1 #or a matrix from igraph networks.
2 as_adjacency_matrix(net, attr="weight")
```

```
17 x 17 sparse Matrix of class "dgCMatrix"

s01  . 22 22 21  .  .  .  .  .  .  .  .  .  .  . 20  .  .
s02 23  . 21  .  .  .  .  .  . 1  5  .  .  .  .  .  .  .
s03 21  .  . 22 1  .  .  4  .  2  1  1  .  .  .  .  .  .
s04  .  . 23  .  .  1  .  .  .  . 22  3  .  .  .  .  2
s05  1 21  .  .  .  .  .  .  2  .  .  .  .  . 21  .  .
s06  .  .  .  .  .  .  .  .  .  .  .  .  .  .  . 21 21
s07  .  .  1  .  .  .  . 22  . 21  .  .  .  4  .  .  .
s08  .  .  2  .  .  . 21  . 23  .  .  .  .  .  .  .  .
s09  .  .  .  .  .  .  .  .  . 21  .  .  .  .  .  .  .
s10  .  .  2  .  .  .  .  .  .  .  .  .  .  .  .  .  .
s11  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .
s12  .  .  .  .  .  2  .  .  .  .  .  . 22 22  .  .  .
s13  .  .  .  .  .  .  .  .  .  .  . 21  .  .  .  .  1
s14                                 1  . 21
```

Turning networks into igraph objects

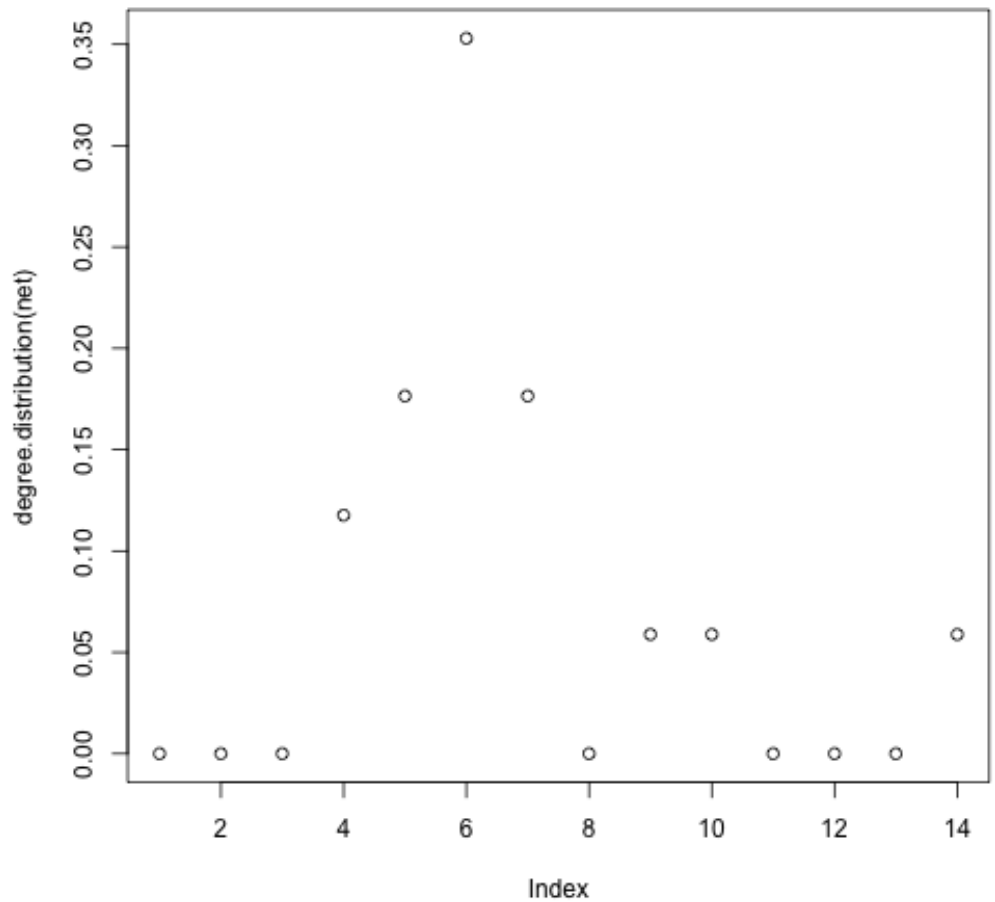
```
# Or data frames describing nodes and edges:
as_data_frame(net, what="edges")
```

	from	to	type	weight
1	s01	s02	hyperlink	22
2	s01	s03	hyperlink	22
3	s01	s04	hyperlink	21
4	s01	s15	mention	20
5	s02	s01	hyperlink	23
6	s02	s03	hyperlink	21
7	s02	s09	hyperlink	1
8	s02	s10	hyperlink	5
9	s03	s01	hyperlink	21
10	s03	s04	hyperlink	22
11	s03	s05	hyperlink	1
12	s03	s08	hyperlink	4
13	s03	s10	mention	2
14	s03	s11	hyperlink	1
15	s03	s12	hyperlink	1

Turning networks into igraph objects

Degree Distribution

```
1 library(igraph)
2 plot(degree.distribution(net))
```



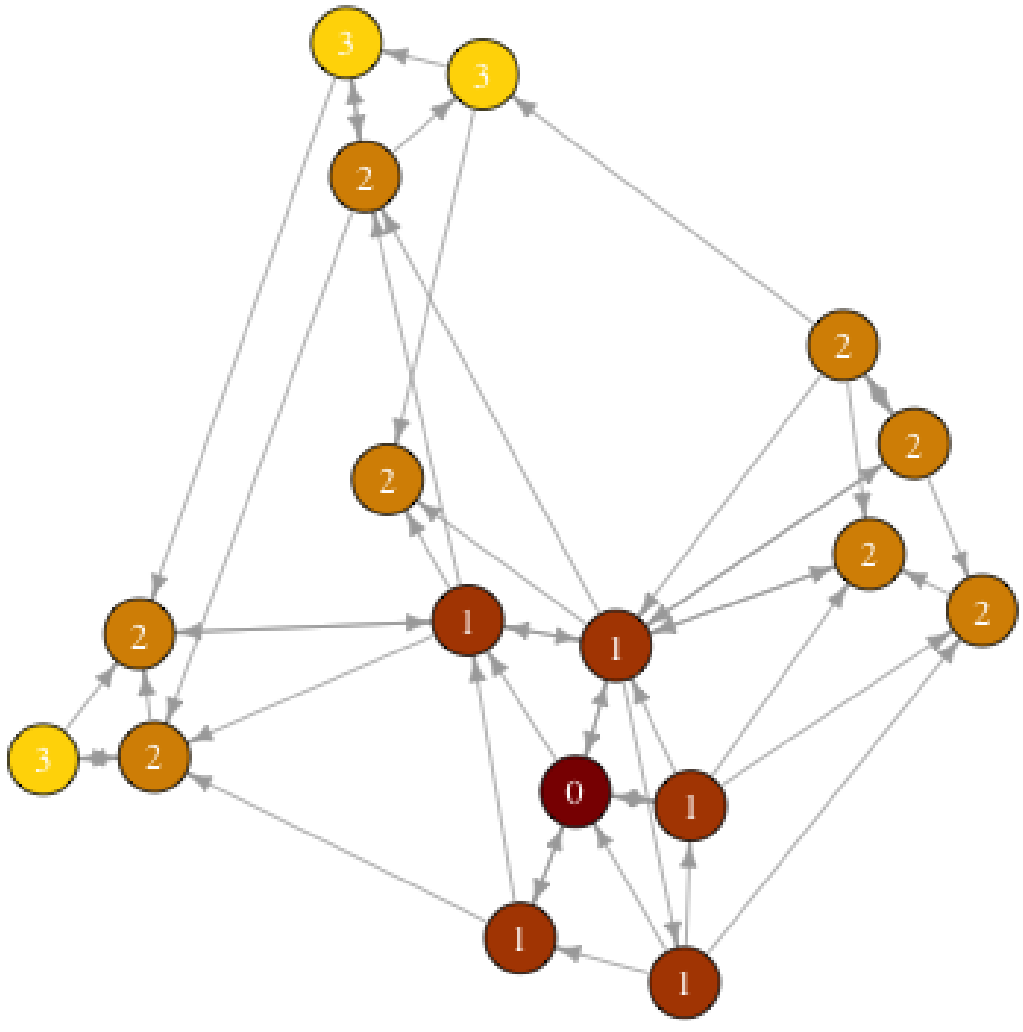
Distances and Paths

- We can extract distances for a node or set of nodes
- Here we get the distance from the New York Times

```
dist.from.NYT = distances(net, v=V(net)[media=="NY Times"],
                          to=V(net), weights=NA)
# Set colors to plot the distances:
oranges <- colorRampPalette(c("dark red", "gold"))
col <- oranges(max(dist.from.NYT)+1)
col <- col[dist.from.NYT+1]

plot(net, vertex.color=col, vertex.label=dist.from.NYT,
     edge.arrow.size=.6, vertex.label.color="white")
```

Distances and Paths

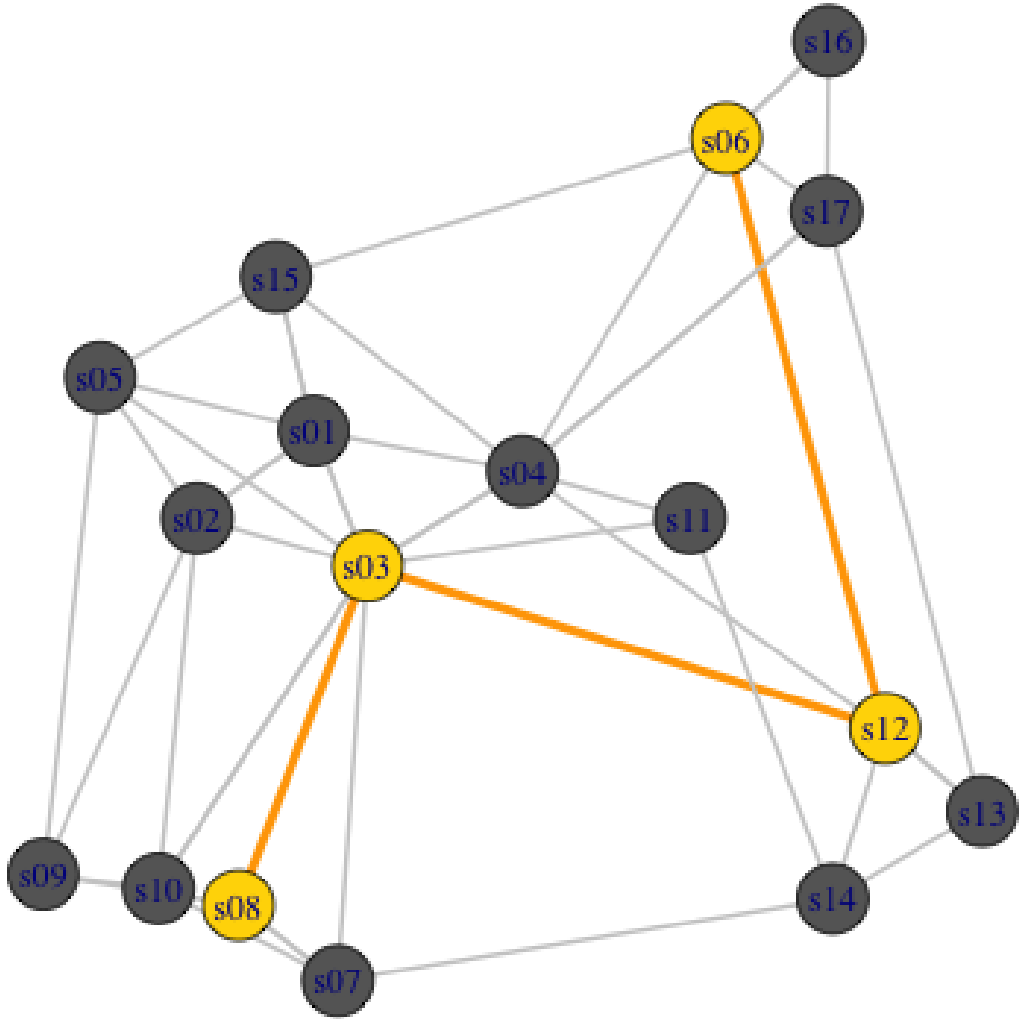


Distances and Paths

- We can also find the shortest path between specific nodes

```
news.path <- shortest_paths(net,
                 from = V(net)[media=="MSNBC"],
                 to  = V(net)[media=="New York Post"],
                 output = "both") # both nodes and edges
# Generate edge color variable to plot the path:
ecol <- rep("gray80", ecount(net))
ecol[unlist(news.path$epath)] <- "orange"
# Generate edge width variable to plot the path:
ew <- rep(2, ecount(net))
ew[unlist(news.path$epath)] <- 4
# Generate node color variable to plot the path:
vcol <- rep("gray40", vcount(net))
vcol[unlist(news.path$vpath)] <- "gold"
plot(net, vertex.color=vcol, edge.color=ecol,
     edge.width=ew, edge.arrow.mode=0)
```

Distances and Paths



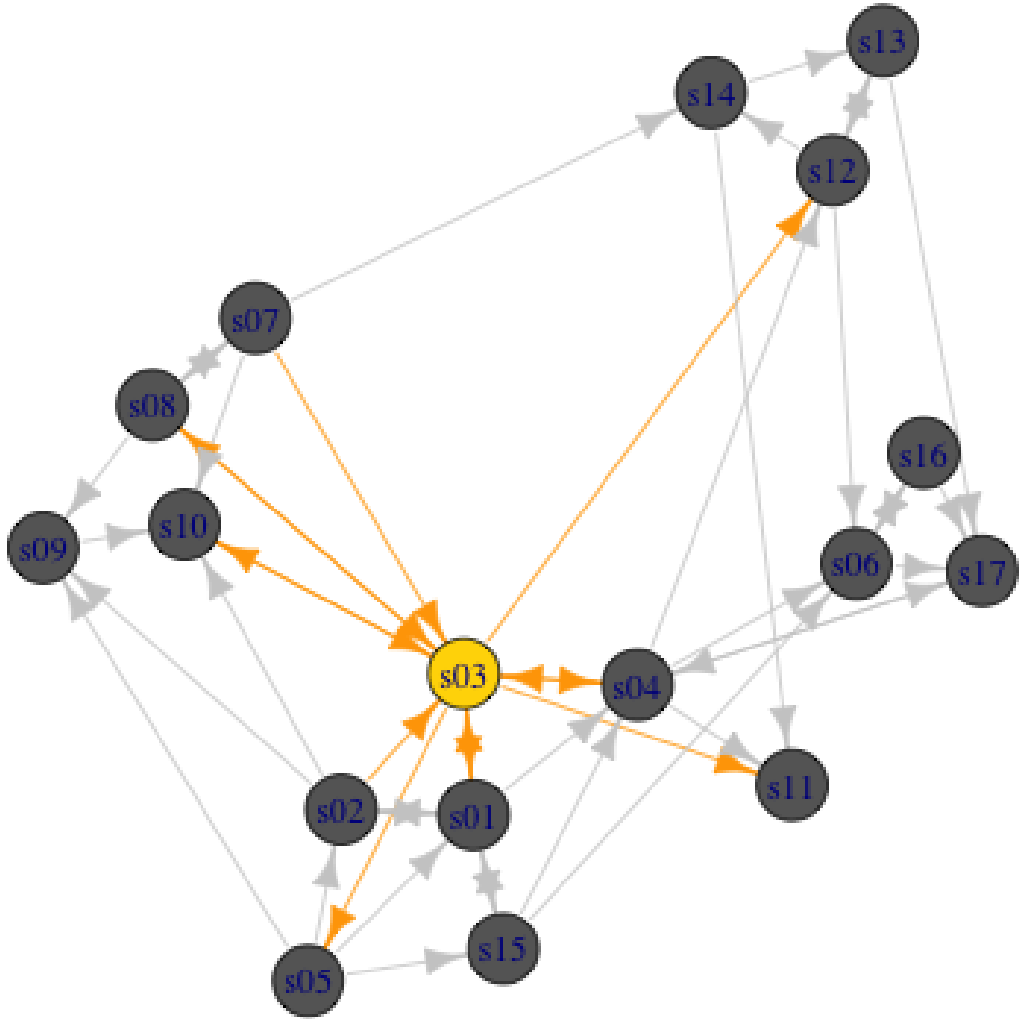
Distances and Paths

- Identify the edges going into or out of a vertex, for instance the WSJ
- For a single node, use 'incident()', for multiple nodes use 'incident_edges()'

```
inc.edges <- incident(net, V(net)
                              [media=="Wall Street Journal"])

# Set colors to plot the selected edges.
ecol <- rep("gray80", ecount(net))
ecol[inc.edges] <- "orange"
vcol <- rep("grey40", vcount(net))
vcol[V(net)$media=="Wall Street Journal"] <- "gold"
plot(net, vertex.color=vcol, edge.color=ecol)
```

Distances and Paths

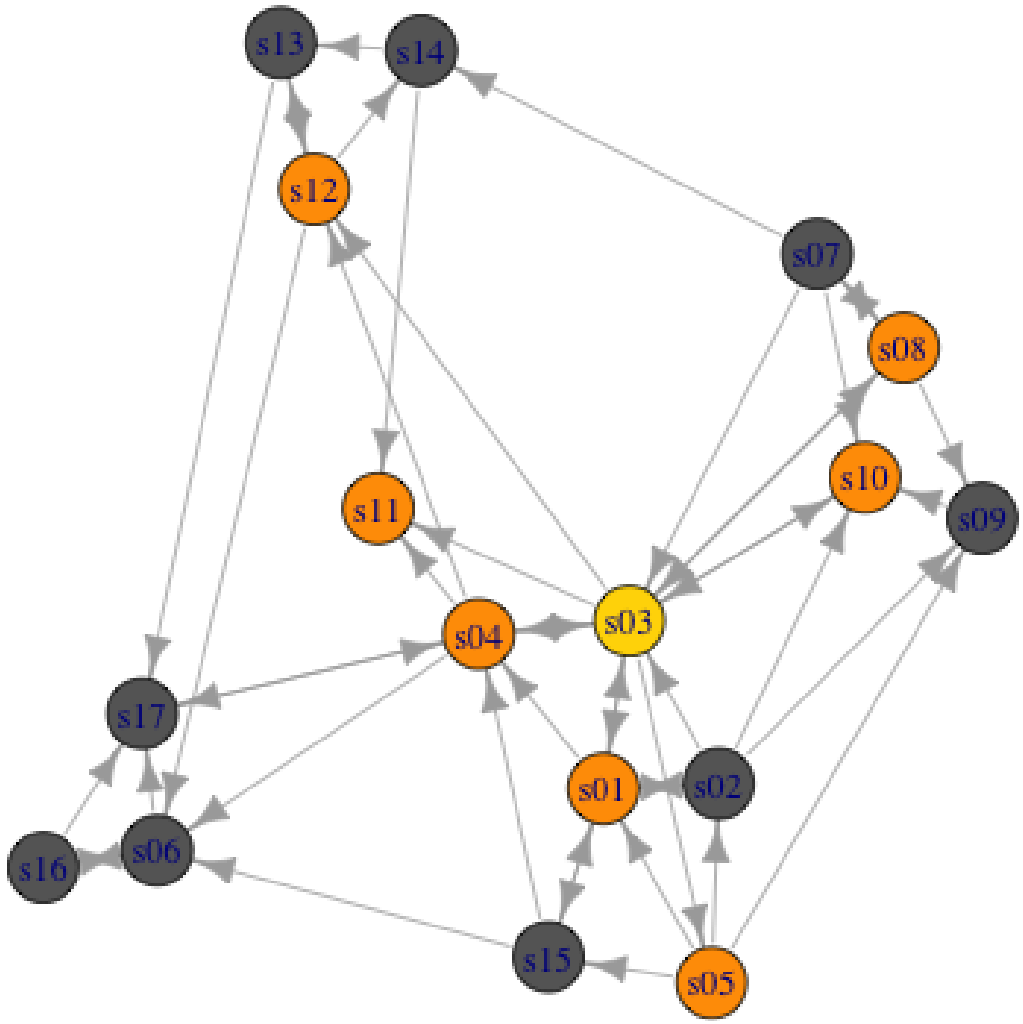


Distances and Paths

- We can also easily identify the immediate neighbors of a vertex, say WSJ
- The 'neighbors' function finds all nodes one step out from the focal actor
- To find the neighbors for multiple nodes, use 'adjacent_vertices()'
- To find node neighborhoods going more than one step out, use function 'ego()' with parameter 'order' set to the number of steps out to go from the focal node(s)

```
neigh.nodes <- neighbors(net, V(net)[media=="Wall Street Jo
# Set colors to plot the neighbors:
vcol[neigh.nodes] <- "#ff9d00"
plot(net, vertex.color=vcol)
```

Distances and Paths



Distances and Paths

- Special operators for the indexing of edge sequences: %–%, %->%, %<-%
- E(network)[X %–% Y] selects edges between vertex sets X and Y, ignoring direction
- E(network)[X %->% Y] selects edges from vertex sets X to vertex set Y
- E(network)[X %->% Y] selects edges from vertex sets Y to vertex set X

Distances and Paths

For example, select edges from newspapers to online sources:

```
E(net)[ V(net)[type.label=="Newspaper"] %->%
V(net)[type.label=="Online"] ]
```

```
+ 7/48 edges from ec5ce29 (vertex names):
[1] s01->s15 s03->s12 s04->s12 s04->s17
[2] s05->s15 s06->s16 s06->s17
```

Distances and Paths

- Two vertices are cocited if there is another vertex citing both of them

```
1 cocitation(net)
```

	s01	s02	s03	s04	s05	s06	s07	s08	s09	s10	s11	s12	s13	s14
s01	0	1	1	2	1	1	0	1	2	2	1	1	0	0
s02	1	0	1	1	0	0	0	0	1	0	0	0	0	0
s03	1	1	0	1	0	1	1	1	2	2	1	1	0	1
s04	2	1	1	0	1	1	0	1	0	1	1	1	0	0
s05	1	0	0	1	0	0	0	1	0	1	1	1	0	0
s06	1	0	1	1	0	0	0	0	0	0	1	1	1	1
s07	0	0	1	0	0	0	0	0	1	0	0	0	0	0
s08	1	0	1	1	1	0	0	0	0	2	1	1	0	1
s09	2	1	2	0	0	0	1	0	0	1	0	0	0	0
s10	2	0	2	1	1	0	0	2	1	0	1	1	0	1
s11	1	0	1	1	1	1	0	1	0	1	0	2	1	0
s12	1	0	1	1	1	1	0	1	0	1	2	0	0	0
s13	0	0	0	0	0	1	0	0	0	0	1	0	0	1

Density

```
net <- graph_from_data_frame(d=links, vertices=nodes,
                             directed=T)
edge_density(net, loops=F)
```

```
[1] 0.1801471
```

```
ecount(net)/(vcount(net)*(vcount(net)-1))
#for a directed network
```

```
[1] 0.1801471
```

Reciprocity

- The proportion of reciprocated ties (for a directed network)

```
reciprocity(net)
```

```
[1] 0.4166667
```

```
dyad_census(net) # Mutual, asymmetric, and null pairs

$mut
[1] 10

$asym
[1] 28

$null
[1] 98
```

```
2*dyad_census(net)$mut/ecount(net) #compute reciprocity
```

```
[1] 0.4081633
```

Clustering Coefficient (or Transitivity)

```
transitivity(net, type="global")
#net is treated as an undirected network
```

```
[1] 0.372549
```

```
transitivity(as.undirected(net, mode="collapse"))
#same as above
```

```
[1] 0.372549
```

```
transitivity(net, type="local")
```

```
 [1] 0.2142857 0.4000000 0.1153846 0.1944444 0.5000000 0.14
 [8] 0.1000000 0.3333333 0.3000000 0.3333333 0.2000000 0.16
[15] 0.3000000 0.3333333 0.2000000
```

Triad Types

Every triple of vertices (A, B, C) are classified into the 16 possible states (Davis & Leinhardt, 1972):

```
# 003  A, B, C, empty graph
# 012  A->B, C
# 102  A<->B, C
# 021D A<-B->C
# 021U A->B<-C
# 021C A->B->C
# 111D A<->B<-C
# 111U A<->B->C
# 030T A->B<-C, A->C
# 030C A<-B<-C, A->C.
# 201  A<->B<->C.
# 120D A<-B->C, A<->C.
# 120U A->B<-C, A<->C.
# 120C A->B->C, A<->C.
# 210  A->B<->C, A<->C.
# 300  A<->B<->C, A<->C, complete graph
```

Triad Types

```
triad_census(net) # for directed networks
```

```
 [1] 244 241  80  13  11  27  15  22   4   1   8   4   4
```

```
length(triad_census(net))
```

```
[1] 16
```

Centrality & Centralization

- Centralization is a general method for calculating a graph-level centrality score based on node-level centrality measure
- The `centr_degree` function returns
  - res (vertex centrality)
  - centralization
  - theoretical_max (maximum centralization score for a graph of that size)

```
1 degree(net, mode="in")
```

```
s01 s02 s03 s04 s05 s06 s07 s08 s09 s10 s11 s12 s13 s14 s15
  4   2   6   4   1   5   1   2   3   4   3   3   2   2   2
```

```
1 centr_degree(net, mode="in", normalized=T)
```

```
$res
 [1] 4 2 6 4 1 5 1 2 3 4 3 3 2 2 2 1 4

$centralization
[1] 0.1948529
```

Closeness

- Closeness (centrality based on distance to others in the graph)
- Inverse of the node's average geodesic distance to others in the network

```
closeness(net, mode="all", weights=NA)
```

```
       s01        s02        s03        s04        s05
0.03333333 0.03030303 0.04166667 0.03846154 0.03225806 0.03
       s08        s09        s10        s11        s12
0.02857143 0.02564103 0.02941176 0.03225806 0.03571429 0.02
       s15        s16        s17
0.03030303 0.02222222 0.02857143
```

```
centr_clo(net, mode="all", normalized=T)
```

```
$res
 [1] 0.5333333 0.4848485 0.6666667 0.6153846 0.5161290 0.50
 [8] 0.4571429 0.4102564 0.4705882 0.5161290 0.5714286 0.43
[15] 0.4848485 0.3555556 0.4571429
```

Betweenness

```
1 betweenness(net, directed=T, weights=NA)
```

```
       s01        s02        s03        s04        s05
24.0000000  5.8333333 127.0000000  93.5000000  16.5000000
       s07        s08        s09        s10        s11
 1.8333333 19.5000000  0.8333333 15.0000000  0.0000000
       s13        s14        s15        s16        s17
20.0000000  4.0000000  5.6666667  0.0000000 58.5000000
```

```
1 edge_betweenness(net, directed=T, weights=NA)
```

```
 [1] 10.833333 11.333333  8.333333  9.500000  4.000000 12.5
 [8]  2.333333 24.000000 16.000000 31.500000 32.500000  9.5
[15] 23.000000 65.333333 11.000000  6.500000 18.000000  8.6
[22] 10.000000  6.000000 11.166667  0.000000 15.000000 21.3
[29]  2.000000  1.333333  4.500000 11.833333 16.833333  6.8
[36] 31.000000 17.000000 18.000000 14.500000  7.500000 28.5
[43] 17.000000  5.666667  9.666667  6.333333  1.000000 15.0
```

Betweenness

```
centr_betw(net, directed=T, normalized=T)
```

```
$res
 [1]  24.0000000   5.8333333 127.0000000  93.5000000  16.50
 [7]   1.8333333  19.5000000   0.8333333  15.0000000   0.00
[13]  20.0000000   4.0000000   5.6666667   0.0000000  58.50

$centralization
[1] 0.4460938

$theoretical_max
[1] 3840
```

Hubs and authorities

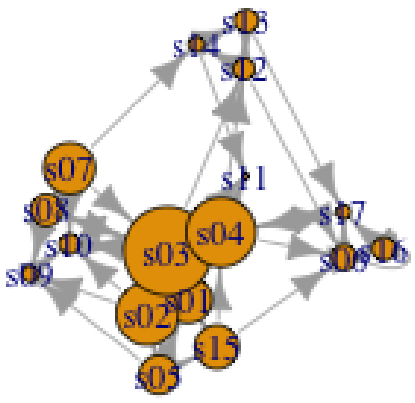
- The hubs and authorities algorithm was developed by Kleinberg to rank web pages
- **Hubs** were expected to contain catalogues with a large number of outgoing links
- **Authorities** would get many incoming links from hubs, presumably because of their high-quality relevant information

```
hs <- hub_score(net, weights=NA)$vector
as <- authority_score(net, weights=NA)$vector

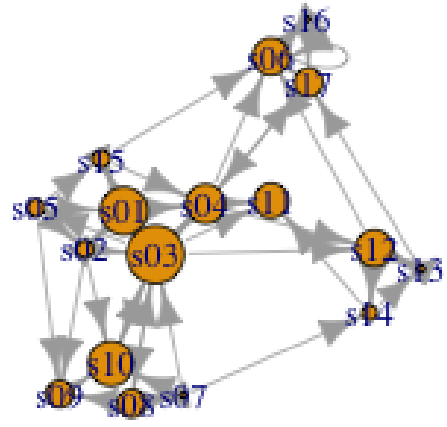
par(mfrow=c(1,2))
 plot(net, vertex.size=hs*50, main="Hubs")
 plot(net, vertex.size=as*30, main="Authorities")
```

Hubs and authorities

Hubs



Authorities



Notes

This presentation is partially based on:

- NetSci X Tutorial "Network Analysis and Visualization with R and igraph" by Katherine Ognyanova, www.kateto.net, NetSciX 2016 School of Code Workshop, Wroclaw, Poland
- Chapter 6 of Arlnold and Tilton, *Humanities Data in R. Exploring Networks, Geospatial Data, Images, and Text.* © 2015 Springer